

Impacto Tecnológico y Social

Desarrollo y proyección de soluciones informáticas modulares y análisis de la Teoría de Juegos mediante el sistema interactivo "Piedra, Papel o Tijera"

Autor del Proyecto David Rafael Dueñas Ripalda

Docente Tutor Mgs. Estefanía Vanessa Heredia
Jiménez

Asignatura & Sede Lógica de Programación -
Ambato 2026

REINVENTEMOS
EL FUTURO

Fundamentos y Teoría

Intersección disciplinar entre el desarrollo de software bajo estándares de ingeniería y la toma de decisiones estratégicas lúdicas.

REINVENTEMOS
EL FUTURO

Teoría de Juegos y Equilibrio

Conflicto de Suma Cero

Desde la perspectiva matemática, el juego representa un escenario clásico de conflicto estricto con información imperfecta y decisiones simultáneas paralelas. Las ganancias de un agente corresponden matemáticamente a las pérdidas exactas del oponente.

Estrategia Mixta Uniforme

Para evitar que un agente humano explote patrones cognitivos o sesgos predecibles en la CPU, el oponente virtual adopta una estrategia mixta. Cada opción debe seleccionarse con una probabilidad idéntica y uniforme del 33.3%.

$$P (Piedra) = P (Papel) = P (Tijera) = \frac{1}{3}$$

REINVENTEMOS
EL FUTURO

Requerimientos del Software

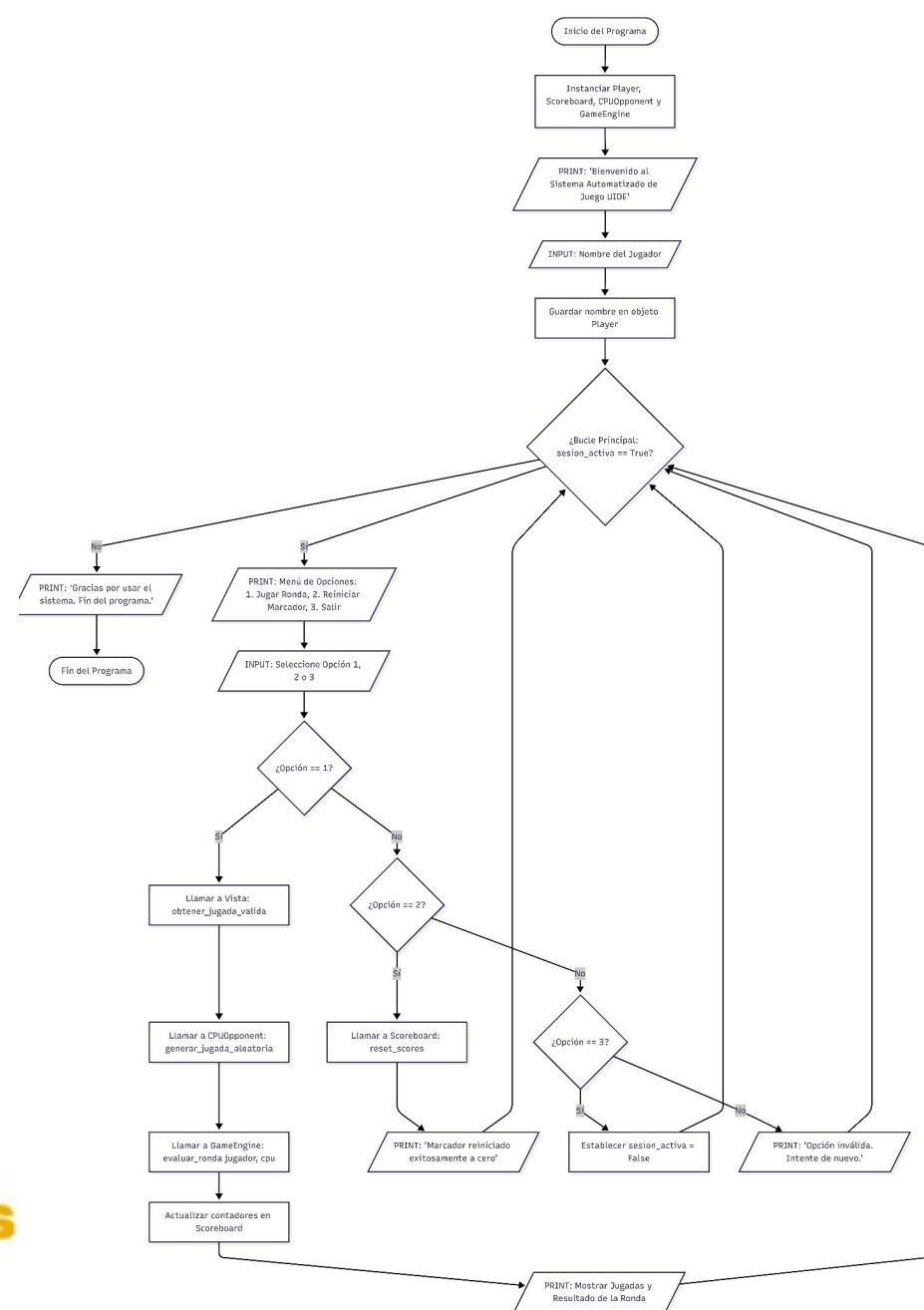
-  **RF-01 (Identidad):** Registro persistente del alias del operario para personalizar todos los mensajes interactivos y reportes lógicos de la sesión activa.
-  **RF-02 (Captura de Entrada):** Filtrado síncrono que restringe la entrada de datos, procesando únicamente jugadas lícitas (Piedra, Papel, Tijera) de forma segura.
-  **RF-03 (Toma de Decisiones):** Ejecución estocástica de la CPU para evitar la predicción de jugadas basada en historial de turnos o sesgos cognitivos.
-  **RNF-01 (Acoplamiento Débil):** Los componentes lógicos que evalúan las reglas del juego y calculan la jugada del CPU deben ser completamente independientes del framework de vista.

REINVENTEMOS
EL FUTURO

Controlador y Flujo Principal

- **Inicialización Atómica:** El programa instancia las variables del Modelo y carga el nombre provisto por el usuario de forma síncrona.
- 🔄 **Bucle de Persistencia:** El ciclo principal se sostiene indefinidamente mediante un control por bandera condicional ('sesion_activa').
- 🎵 **Orquestación de Opciones:** Derivación automática según la selección del menú: Jugar Ronda, Reset de Marcadores o Salida del sistema.

REINVENTEMOS
EL FUTURO

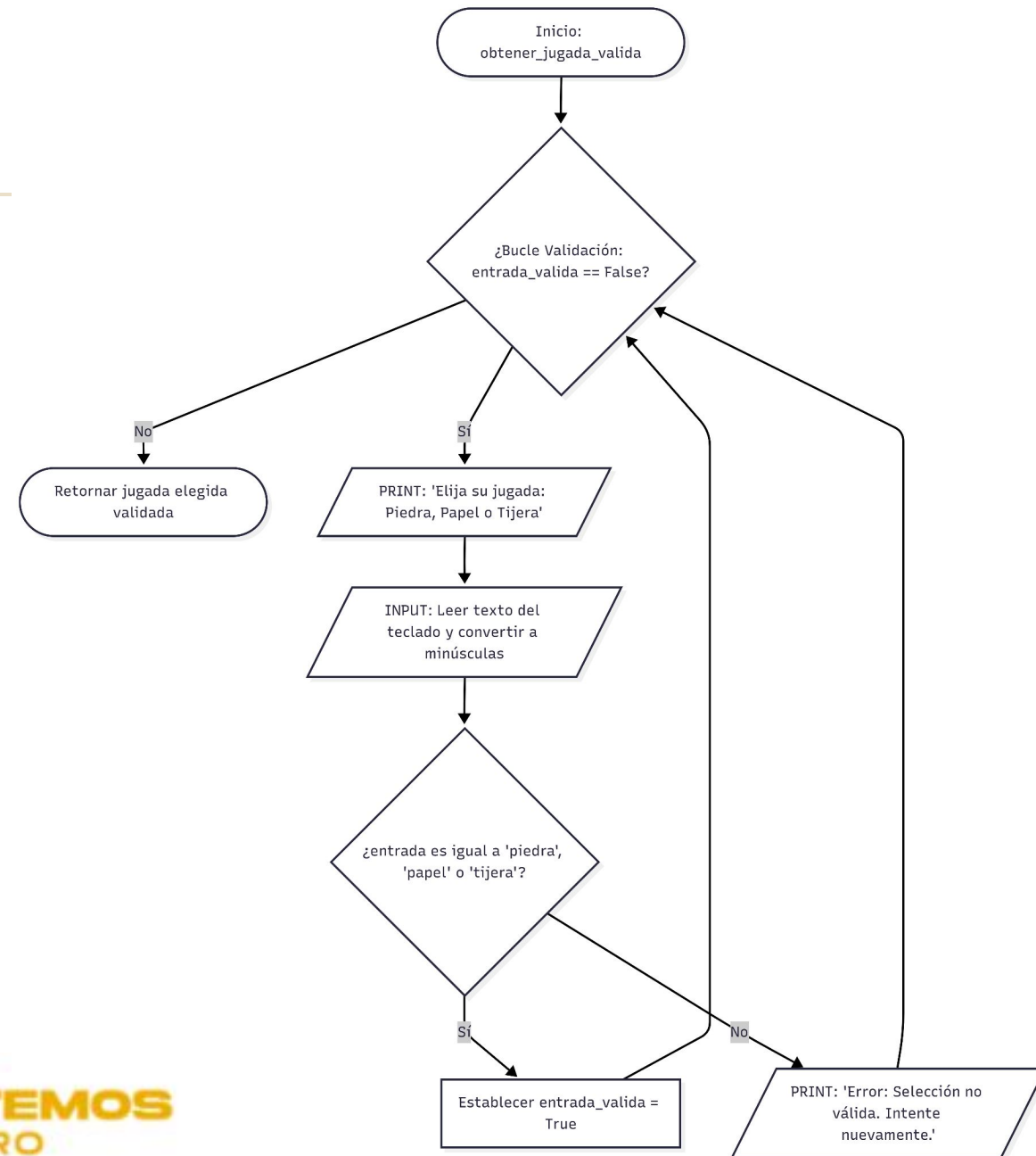


Capa de Vista y Validación

Bucle de Validación de Entradas: Se ejecuta una subfunción exclusiva en la interfaz física que no permite avanzar el flujo lúdico ante comandos corruptos o erróneos.

Normalización de Strings: El sistema convierte síncronamente a minúsculas la entrada de texto del operario para evitar fallas sintácticas de coincidencia.

Control de Excepciones: En caso de que el valor no sea válido, imprime un mensaje de alerta en consola y repite el ciclo interactivamente.



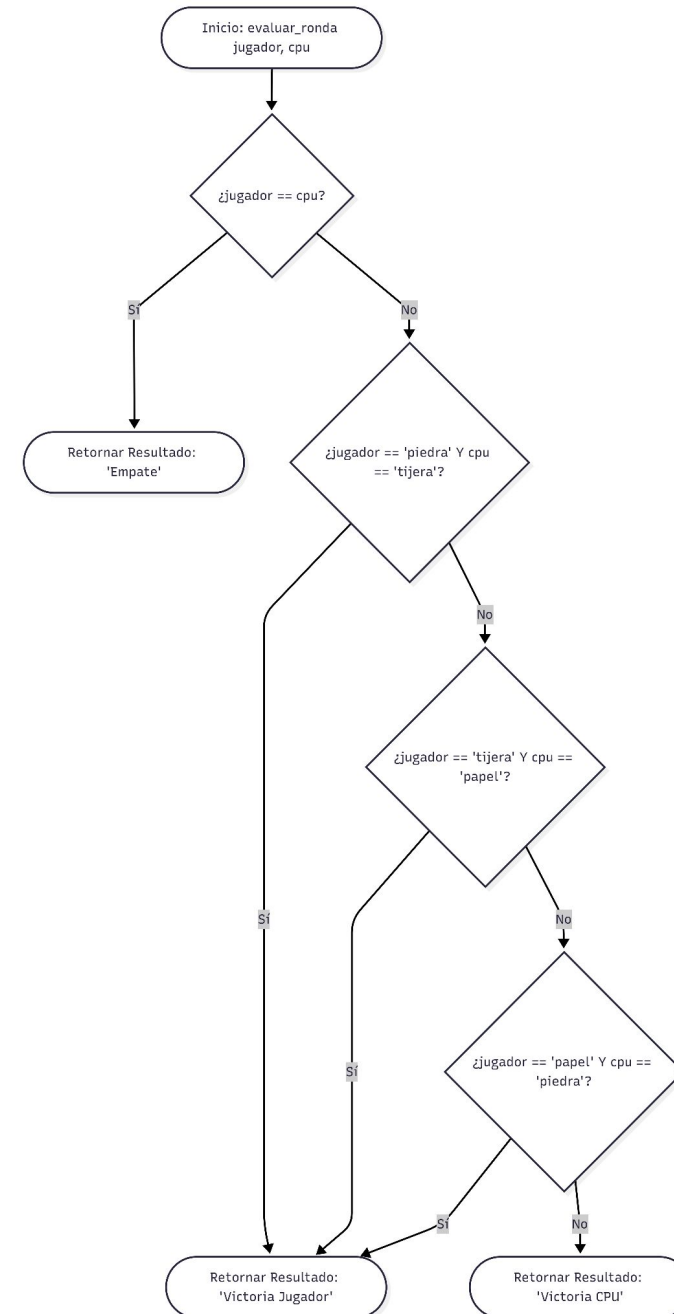
REINVENTEMOS
EL FUTURO

Reglas de Negocio del Modelo

 **Detección Inmediata de Empates:** Compara directamente el estado de ambas elecciones, retornando un empate si las cadenas de caracteres son equivalentes.

 **Árbol de Decisiones de Victoria:** Evalúa estrictamente las tres combinaciones condicionales que dan la victoria al humano: Piedra vs Tijera, Tijera vs Papel y Papel vs Piedra.

 **Resolución por Descarte:** Si el enfrentamiento no es empate ni cumple las condiciones humanas, el motor lógico declara la ronda ganadora a la CPU.



REINVENTEMOS
EL FUTURO

Arquitectura de Capas MVC



Modelo (Dominio)

Mantiene el estado interno persistente temporal (Scoreboard) y ejecuta la lógica pura del dominio matemático probabilístico, aislado de la interfaz física.



Vista (Presentación)

Gestiona exclusivamente los flujos de Entrada y Salida (E/S), capturando los strings del operario y renderizando el estado procesado en consola.



Controlador

Orquesta el ciclo de vida del software, atrapando las elecciones del usuario y despachando modificaciones de estado atómicas al Modelo.

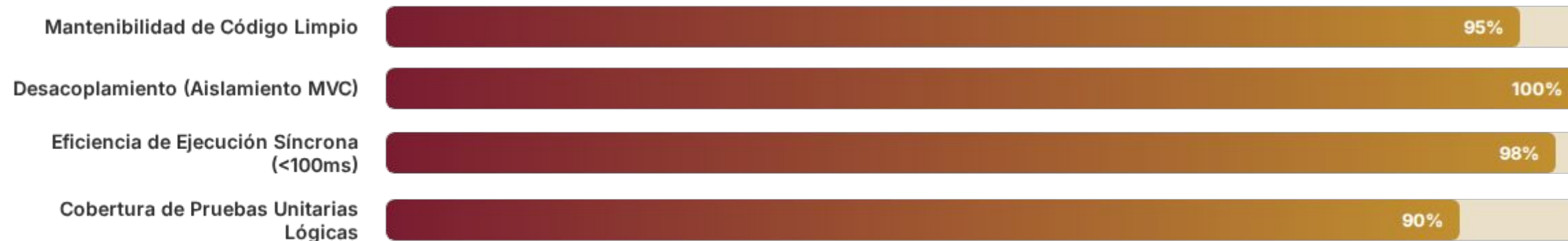
REINVENTEMOS
EL FUTURO

Trazabilidad de Requisitos

Código Requisito	Descripción Funcional	Archivo y Método del Código Fuente	Flujograma de Validación Asociado
RF-01 (Identidad)	Captura e imprime el alias para personalizar la sesión activa.	`Player` (Constructor)	`Automated Game System`
RF-02 (Filtro)	Filtra entradas restringiendo a Piedra, Papel o Tijera.	`ConsoleInterface.obtener_jugada_valida()`	`Jugada Valida Flow`
RF-03 (CPU)	Generación aleatoria uniforme de la CPU sin sesgos cognitivos.	`CPUOpponent.generar_jugada()`	`Automated Game System`
RF-04 (Evaluación)	Mapeo de la matriz de pagos: Victoria, Derrota o Empate.	`GameEngine.evaluar_ronda()`	`Jugador vs CPU Ronda`



Métricas de Calidad de Software



Las pruebas demuestran que aislar el núcleo probabilístico del juego acelera los tiempos de respuesta y garantiza un acoplamiento nulo ante variaciones físicas en la capa de visualización.

REINVENTEMOS
EL FUTURO

Ética e Impacto Tecnológico

//

La ingeniería de software aplicada a la sociedad moderna no se limita a resolver problemas operativos; su verdadero impacto reside en asegurar la equidad algorítmica y la transparencia matemática en cada sistema automatizado interactivo.

//

— David Rafael Dueñas Ripalda,
2026

REINVENTEMOS
EL FUTURO